# ISO 15939 Measurement Process Simulator

Java Swing GUI Application

*Course: Software Project II*

*Assignment Type: Individual Project*

## 1. Purpose

In this assignment, you will develop a Java Swing desktop application that simulates the 5 core steps of the ISO/IEC 15939 software measurement process standard. The application will enable defining quality dimensions and metrics of a software system, collecting data, and analyzing the results.

Review the web application to understand the workflow. The Java version will implement the same logic in a GUI environment.

## 2. About ISO/IEC 15939

ISO/IEC 15939 is an international standard that formalizes the measurement process in software engineering. The standard defines a systematic process from identifying information needs to defining metrics, collecting data, and performing analysis. The 5 steps in this assignment represent a simplified model of the standard's measurement process:

| Step | Name | Description |
|---|---|---|
| 1 | **Profile** | User and session information is entered |
| 2 | **Define** | Quality type (Product/Process), mode (Custom/Health/Education), and scenario are selected |
| 3 | **Plan** | Dimensions and metrics belonging to the selected scenario are displayed (coefficient, direction, range, unit) |
| 4 | **Collect** | Raw data values are shown for each metric; a score between 1–5 is calculated |
| 5 | **Analyse** | Dimension-based weighted averages are calculated; a radar chart and gap analysis are presented |

## 3. Technical Requirements

### 3.1. Technologies to Use

- Java SE 17 or higher
- Java Swing (JFrame, JPanel, JButton, JLabel, JTextField, JComboBox, JTable, JRadioButton, etc.)
- Collections Framework (ArrayList, HashMap)
- OOP principles (Encapsulation, Inheritance, Polymorphism)

### 3.2. Required Design Patterns

- Wizard structure using CardLayout — each step must be designed as a separate JPanel
- Model classes and GUI classes must be separated from each other (MVC pattern)
- All scenario data may be defined hard-coded inside Java classes (file reading is not required)

## 4. Application Steps (Detailed)

### 4.1. Step 1: Profile

This is the entry screen that collects user information.

#### Required Fields:

- **Username**
- **School**
- **Session Name**

#### Validation:

If the "Next" button is clicked before all fields are filled in, a warning message must be displayed. The warning messages should be user-friendly: clearly describe which field is missing, use plain language that avoids technical jargon, and guide the user on exactly what action is needed to proceed (e.g., "Please enter your username to continue." or "All fields must be filled in before proceeding to the next step.").

### 4.2. Step 2: Define Quality Dimensions

This is the screen where the user determines the scope of measurement. A three-stage selection is made:

#### 2a. Quality Type Selection:

The user must be able to select exactly one quality type at a time. Consider which Swing component best enforces the constraint that only one option can be active simultaneously — your choice of component should reflect this mutual exclusivity requirement.

| Type | Description |
|---|---|
| **Product Quality** | Software product characteristics: performance, security, usability, reliability |
| **Process Quality** | Development process characteristics: sprint efficiency, code quality, team collaboration |

### 2b. Mode Selection:

Similar to the quality type selection, the user should only be able to activate one mode at a time. Choose an appropriate component that enforces single-selection behavior among the available modes.

| Mode | Description |
|---|---|
| **Custom** | User defines their own dimensions and metrics from scratch (Bonus feature) |
| **Health** | Health management system scenarios (ready-made dataset) |
| **Education** | Education LMS system scenarios (ready-made dataset) |

### 2c. Scenario Selection:

Apply the same single-selection logic described above: use a component that naturally prevents multiple scenarios from being selected at once, reflecting the fact that each session is associated with exactly one scenario.

Different scenarios are listed depending on the selected mode. For example, for Education mode: "Scenario C — Team Alpha" and "Scenario D — Team Beta".

*Note: A minimum of 2 different modes must be defined, and at least 2 scenarios must be defined for each mode.*

## 4.3. Step 3: Plan Measurement

The pre-defined quality dimensions and metrics belonging to the selected scenario are displayed in table form. This step is read-only.

### Information to be displayed for each dimension:

- Dimension name and its coefficient — e.g., Usability (Coefficient: 25)

### Columns to be displayed for each metric:

| Column | Description |
|---|---|
| **Metric** | Metric name (e.g., SUS score, Onboarding time) |
| **Coefficient** | Weight of the metric within the dimension (e.g., 50) |
| **Direction** | "Higher is better" or "Lower is better" |
| **Range** | Valid value range (e.g., 0–100) |
| **Unit** | Unit (e.g., points, minutes, seconds, %, users) |

### Minimum Dataset Example (Education — Scenario C):

The table below is a sample dataset you may use in your application. You can create your scenarios according to this structure:

| Dimension | Metric | Coeff | Direction | Range | Unit |
|---|---|---|---|---|---|
| Usability (25) | SUS score | 50 | Higher ↑ | 0–100 | points |
| | Onboarding time | 50 | Lower ↓ | 0–60 | min |
| Perf. Efficiency (20) | Video start time | 50 | Lower ↓ | 0–15 | sec |
| | Concurrent exams | 50 | Higher ↑ | 0–600 | users |
| Accessibility (20) | WCAG compliance | 50 | Higher ↑ | 0–100 | % |
| | Screen reader score | 50 | Higher ↑ | 0–100 | % |
| Reliability (20) | Uptime | 50 | Higher ↑ | 95–100 | % |
| | MTTR | 50 | Lower ↓ | 0–120 | min |
| Func. Suitability (15) | Feature completion | 50 | Higher ↑ | 0–100 | % |
| | Assignment submit rate | 50 | Higher ↑ | 0–100 | % |

## 4.4. Step 4: Collect Data

In this step, the raw data values (value) collected for each metric are displayed and a score between 1–5 is automatically calculated from these values. Data may be hard-coded according to the scenario.

### Score Calculation Formula:

**"Higher is better" metrics:** `score = 1 + (value − min) / (max − min) × 4`

**"Lower is better" metrics:** `score = 5 − (value − min) / (max − min) × 4`

The result must be between 1.0 and 5.0 and must be rounded to the nearest 0.5.

### Table View :

| Metric | Direction | Range | Value | Score (1–5) | Coeff / Unit |
|---|---|---|---|---|---|
| SUS score | Higher ↑ | 0–100 | 89 | **5** | 50 / points |
| Onboarding time | Lower ↓ | 0–60 | 5 | **5** | 50 / min |

### 4.5. Step 5: Analyse

This is the results screen where the collected data is analyzed. It must include the following three components:

#### 5a. Dimension-Based Weighted Average:

The weighted average of the metrics is calculated for each dimension:

```
dimensionScore = Σ(metricScore × metricCoefficient) / Σ(metricCoefficient)
```

The score and name of each dimension must be displayed using a JProgressBar or a colored bar.

#### 5b. Radar Chart (Bonus):

A radar (spider/pentagon) chart showing the scores of all dimensions must be drawn. It can be implemented using Java 2D Graphics (Graphics2D). This section will be evaluated as a bonus.

#### 5c. Gap Analysis:

The dimension with the lowest score is identified and the following information is displayed:

- Dimension name and score
- Gap value (5.0 − score)
- Quality level label (Excellent / Good / Needs Improvement / Poor)
- "This dimension has the lowest score and requires the most improvement."

## 5. Suggested Class Structure

A well-structured class hierarchy is expected for this project. While the specific implementation is left to you, keep in mind that a clean separation of concerns will be evaluated: think about how you would organize classes so that data/logic and interface are not mixed together, how you would represent individual metrics and their groupings, and how you would manage scenario data centrally. Your design should reflect solid OOP principles — encapsulation, inheritance, and appropriate use of polymorphism.

## 6. Step Indicator

A step indicator showing the step numbers and their status, similar to the web application, must be present at the top of the screen:

- The active step must be highlighted (different color or bold text)
- A ✓ (check) mark must be shown on completed steps
- Step names: Profile → Define → Plan → Collect → Analyse

## 7. Evaluation Criteria

| Criterion | Weight | Points |
|---|---|---|
| OOP usage (class design, inheritance, encapsulation) | 20% | /20 |
| Collections usage (ArrayList, HashMap) | 10% | /10 |
| Swing GUI quality (layout, components, visual consistency) | 20% | /20 |
| Complete implementation of all 5 steps | 25% | /25 |
| Score calculation accuracy and gap analysis | 15% | /15 |
| Code readability and comments | 10% | /10 |
| **Radar Chart (Bonus)** | **+10%** | **/10** |
| **TOTAL** | **100% (+10)** | **/110** |

## 8. Submission Requirements

1. The project must be compilable and runnable from the command line using javac and java commands. There must be no IDE dependency.
2. No external libraries may be used — only the standard Java SE library.
3. The README.txt file must include: student name, student ID, compilation/run instructions, and a screenshot.
4. Plagiarism will be checked using Turnitin and a diff tool. Assignments that are identical to each other will receive a grade of 0 (zero).
5. You must push your project to a public GitHub repository. The repository must contain your complete source code and commit history.

**Good luck!** *For your questions, you may reach out during class hours or via email.*